ONSITE STORAGE · OSS-NEXT · DEVELOPMENT UPDATE

Dev-only admin: SEO Page Configurator & homepage Content Editor

Everything landed on Wednesday, 5 August 2026.

Date Wed 5 Aug 2026, 16:31 (+08:00) **Branch** integrate-wp-render **Commit** `357ccb0` **Author** 1997xjustin12

One commit shipped that day, and it was a large one: a self-contained admin area that lets non-developers change page SEO and homepage copy without a code change or a redeploy. It is reachable in development and preview builds only, and is blocked three separate ways in production.

1	60	+2,581 / −239	2	16
COMMIT	FILES TOUCHED	LINES CHANGED	NEW ADMIN TOOLS	ROUTES NOW SEO-EDITABLE

What shipped

1 · Page Configurator — per-page SEO overrides

Edit meta title, description, keywords, canonical URL, robots directives, Open Graph fields and per-page scripts for each of the app's 16 static routes.

- Every page's shipped copy moved into `config/pageSeoDefaults.ts`, so there is a single source of truth: the page renders from it, and the admin shows it as the placeholder behind each field.
- A blank field means **"leave alone"** — never "blank the tag". Clearing a field cannot accidentally strip a live meta tag.
- Scripts are structured entries rendered through `next/script`. The `beforeInteractive` strategy is deliberately not offered, because Next.js only honours it from the root layout.

2 · Content Editor — homepage headings

All 17 heading elements on the live homepage are now editable.

- Headings with styled spans use a `[[accent]]` marker instead of raw HTML, so an edit cannot inject markup or break the layout — while the design's emphasis survives the edit.
- The three Hero layout variants share one set of headings. Their wording is identical and only the layout differs, which keeps the A/B test measuring one thing.
- Saves store **only** headings that differ from source, so editing a default still reaches pages nobody has customised.

3 · Hero variant preview

`/?variant=N` (dev and preview only) renders a specific Hero variant on demand. Assignment was random on first visit then pinned for 30 days, which left no way to review the other two designs. The preview deliberately does **not** write the cookie — previewing must not change which variant a visitor is enrolled in.

Storage — one Redis instance, four storefronts

Both tools are backed by Redis and store per-store records, so a single Upstash instance can serve all four storefronts without collision:

```
oss-next:<store>:seo:<path>        ← Page Configurator
oss-next:<store>:content:<path>    ← Content Editor
```

The store slug comes from a new `NEXT_PUBLIC_STORE_KEY` (`config/store.ts`), defaulting to `onsite`. Deriving it from `NEXT_PUBLIC_STORE_DOMAIN` was rejected: that value is empty on localhost, which would silently point a dev box at the wrong store's keys.

Production gate — three layers

Each layer covers a hole the others do not. The admin has **no authentication**; these layers are what keeps it unreachable.

LAYER	WHAT IT COVERS
`proxy.ts`	404s the `/admin` prefix — the path users actually hit, and the only layer that returns a real 404 status.
`app/admin/layout.tsx`	Refuses to render the admin shell at all.
`Every Server Action`	Re-checks independently — a form's POST target is its own endpoint, so a layout gate does not cover it.

> **Why the layout returns a dead page instead of calling `notFound()`:** under `cacheComponents` these routes prerender at build time, and throwing there dumped a 404 stack trace for every admin route into every production build.

New building blocks

PATH	PURPOSE
`app/admin/`	Admin shell, nav, Page Configurator and Content Editor routes
`actions/seo.ts · actions/content.ts`	Save / reset Server Actions, each with its own production check
`services/seo.service.ts` `services/content.service.ts`	Redis reads/writes behind `'use cache'`
`config/pageSeoDefaults.ts`	Shipped SEO copy for all 16 routes — one source of truth
`config/homeContent.ts`	Default homepage headings
`config/pages.ts · config/store.ts · config/admin.ts`	Route registry, store key, admin config
`lib/admin.ts`	Shared gate logic used by all three layers
`lib/seo.ts · lib/content.ts`	Merge overrides over defaults at render time
`components/shared/AccentText.tsx`	Renders the `[[accent]]` marker as a styled span
`components/shared/PageHeadScripts.tsx`	Renders configured scripts via `next/script`
`types/seo.ts · types/content.ts`	Record shapes for both tools

Sixteen existing `page.tsx` files and nine homepage components were updated to read from the new defaults/override layer. `config/cache.ts` gained the `seo` tag so saves can bust the right cache. The former `app/(admin)/` route group was renamed to `app/admin/` so the URL prefix is real and can be blocked at the proxy.

Verification

✓ `tsc`, `eslint` and `next build` all clean.

✓ Playwright round-trips through both admin forms: save, reset, and live storefront update via `updateTag`.

✓ Accent spans confirmed intact after editing.

✓ All three Hero variants confirmed picking up edited headings.

Before this goes anywhere near production: the admin has no authentication of any kind. Real auth must be added before any of the three gate layers is relaxed.